TEK-UP
Ecole Supérieure Privée
Technologie & Ingénierie

Rapport de Test

Réalisé par : RIAHI arbia

Classe :GLSI -5-A

Table des matières

I. Conception des tests : 4
1. stratégie de test 4
1.1. Approche 4
1.2. Méthodologie : 5
2. Outils des tests 6
II. Scénarios de test 8
III. Couverture des tests - Approche et Méthodes de mesure 11
1. Approche utilisée 11
2. Mesure de la couverture 12
IV. Analyse des resultats Couverture des tests 15
1. Analyse de la couverture du code (JaCoCo – White Box) 16
2. Analyse des tests fonctionnels (ExtentReport – Selenium – Black Box) 18

3. Analyse de la sécurité (Snyk) 19

4. Analyse qualité SonarQube 20

I. Conception des tests :

La conception des tests repose sur une collaboration étroite entre le PO, les développeurs et l'ingénieur QA. Dès les premières phases du sprint, les exigences sont clarifiées, analysées et transformées en scénarios de test fonctionnels et techniques. Les cycles "**3 Amigos**" garantissent l'alignement sur les critères d'acceptation, la bonne compréhension métier et la faisabilité technique.

1. stratégie de test

1. 1. Approche

Le projet adopte une approche **hybride** mélangeant tests manuels et automatisés afin d'obtenir une couverture optimale :

- Les tests manuels seront utilisés pour les cas exploratoires, les validations visuelles et les scénarios à faible fréquence.
- L'automatisation sera priorisée *pour les régressions*, les cas critiques, et les tests répétitifs.
- Les tests se focalisent sur *les tâches répétitives* opérées par le client sur l'application.

1. 2. Méthodologie :

Une méthodologie Agile/Scrum sera suivie, intégrant les tests dans chaque sprint :

- Les activités QA sont planifiées dès les phases de conception
- Rédaction de scénarios en parallèle des spécifications
- Revue croisée avec les développeurs et PO, incluant notamment la réunion des **3 Amigos** pour aligner compréhension et critères d'acceptation avant développement
- Intégration continue (CI) avec déclenchement des tests automatisés à chaque build
- Déploiement continu avec validation des livraisons en pré-production.

2. Outils des tests

- ✓ **Outil de gestion des tests:**

  L'outil choisi pour la gestion des cas de test, de leur exécution et du suivi des résultats est Xray jira. Il permet une traçabilité complète et un accès collaboratif aux informations.

- ✓ **Outil de gestion des anomalies:**

  Les anomalies sont enregistrées, suivies et analysées via Jira. Cet outil facilite la communication entre les équipes et assure le suivi du cycle de vie des défauts.

- ✓ **Automatisation fonctionnelle :**

- ✓ Les tests fonctionnels sont automatisés avec **Cucumber** en suivant l'approche BDD, permettant de lier les scénarios en **Gherkin** aux exigences métier. Les étapes sont codées en **Java** dans les Step Definitions, avec **Selenium WebDriver** pour simuler les actions utilisateur. **JUnit** est utilisé pour organiser et exécuter les tests, avec des annotations (@Before, @After, @Test, etc.) pour gérer le flux de test de manière modulaire. L'intégration avec

Jenkins permet une exécution automatisée via des pipelines CI/CD, garantissant une validation continue et une détection rapide des régressions.

- ✓ **tests de performance:**

  Les tests de performance sont effectués avec *JMeter* pour simuler des charges et analyser la réponse du système.

- ✓ **Outil de tests de sécurite et audit :** Snyk (SAST + SCA + container scanning)
- ✓ **Analyse de code :** SonarQube

II.Scénarios de test

Les scénarios de test sont rédigés et organisés dans Google Sheets pour assurer une structuration claire

		G Gestion Utilisateurs			
TS_004	Création utilisateur	vérifier la creation d'un utilsateur	Assurer un temps de création minimal même avec un volume important de requêtes.	P1	22
TS_005	Modification utilisateur	vérifier la modification d'un utilisateur	Garantir une mise à jour fluide avec un temps de réponse constant.	P4	24
TS_006	Pagination des utilisateurs	Vérifier navigation page	Maintenir une navigation fluide même avec un très grand nombre utilisateurs	P1	37
TS_007	Archivage utilisateur	vérifier l'archivage d'un utilisateur	Assurer une traçabilité complète et fiable des opérations dans la base.	P1	9
TS_008	Recherche utilisateur	Vérifier la recherche par nom, email, rôle ou matricule	Fournir une recherche rapide malgré la diversité des critères et la taille des données.	P4	21
		Gestion actualités			
TS_009	Création actualité	vérifier la creation d'une actualité	Assurer une création rapide, même avec des contenus volumineux.	P1	33
TS_010	Modification actualité	vérifier la modification d'une actualité	Garantir une mise à jour instantanée quelle que soit la charge.	P1	20
TS_011	Supprimer une actualité	vérifier la suppression d'une actualité	Assurer une suppression rapide et sans impact sur les performances globales.	P2	9
TS_012	Pagination des actualités	navigation entre les pages	Maintenir une navigation fluide entre les pages actualités, même en masse.	P3	13

Figure 1 : Scénarios de test pour les modules “Gestion des utilisateurs” et “Gestion des actualités”

TS_ID	Titre du scénario	Description Fonctionel	description non fonctionel	Priorité	Nb estimé de cas de tests
TS_001	Vérifier la connexion	Vérifier la connexion pour chaque rôle	Assurer une connexion rapide et protégée contre toute tentative non autorisé.	P0	23
TS_002	Vérifier la déconnexion	vérifier la déconnexion pour chaque role	Garantir une déconnexion immédiate et invalider toute tentative de réutilisation URL précédente.	P0	11
TS_003	Récupération du mot de passe	verifier la recuperation de mot de passe par email	Sécuriser le lien de réinitialisation et assurer un envoi email performant même sous charge.	P1	20

Figure 2: Scénarios de test pour les modules “Connexion”, “Déconnexion” et “mot de passe oublié”

Gestion Profile					
TS_013	Changement de mot de passe	vérifier le changement de mot de passe	Garantir un changement sécurisé du mot de passe.	P3	13
TS_014	Informations du compte	vérifier la consultation de profile	Assurer une mise à jour instantanée des informations du profil.	P3	21
TS_015	Changement des informations personelles	vérifier le changement des informations personelles	Tracer et auditer chaque modification liée à la sécurité du compte	P3	12
Gestion Projet					
TS_016	Liste des projets	Vérifier affichage de la liste des projets	Afficher la liste des projets de manière fluide même avec plusieurs centaines d'enregistrements.	P3	8
TS_017	Détails projet	Vérifier affichage des détails projet	Assurer un affichage immédiat et stable des informations détaillées.	P3	11
TS_018	Création projet	Vérifier ajout projet	Garantir une création rapide et sans latence notable.	P3	10
TS_019	Modification projet	Vérifier la mise à jour projet	Assurer une mise à jour performante quel que soit le volume de données.	P3	17
TS_020	Recherche de projets	Vérifier la recherche projet	Maintenir des temps de recherche rapides sur de gros volumes.	P3	11
TS_021	Archivage de projet	Vérifier archivage projet	Assurer une opération d archivage fiable et rapide.	P3	9
TS_022	Pagination des projets	Vérifier la pagination des projets	Garantir une transition instantanée entre les pages de projets.	P4	29
TS_023	Rapport de projet	Vérifier la génération du rapport de projet	Générer les rapports rapidement sans dégrader les performances.	P4	13
Gestion Taches					
TS_024	Création tâche	Vérifier ajout tache	Assurer une création de tâche instantanée même sous forte sollicitation.	P4	13
TS_025	Modification tâche	Vérifier la mise à jour tâche	Garantir une mise à jour performante quel que soit le volume	P4	11
TS_026	Archivage tache	Vérifierarchivage tache	Assurer une archivage fluide et entièrement traçable.	P4	16
TS_027	Statut tache	Vérifier la mise à jour du statut tâche	Maintenir une mise à jour du statut en temps réel sans latence.	P4	22
TS_028	Pagination des tâches	Vérifier la pagination des tâches	Assurer une navigation fluide entre les pages de tâches.	P2	3
Notifications					
TS_029	Consultation des notifications	Vérifier la consultation des notifications	Garantir un affichage instantané des notifications, même nombreuses.	P4	13
TS_030	Notifications en temps réel	Vérifier la réception en temps réel des notifications	Assurer une réception immédiate et fiable des notifications en direct.	P4	11
chat Interne					
TS_031	Chat interne	Vérifier envoi et la réception des messages en temps réel	Assurer échange instantané et fiable des messages en temps réel.	P4	16

Figure 3 : Scénarios de test pour les modules “Profil utilisateur”, “Projets”, “Tâches”, “Notifications” et “Messagerie interne”

III. Couverture des tests - Approche et Méthodes de mesure

1. Approche utilisée

Tests boîte blanche (White Box Testing)

L'approche boîte blanche consiste à écrire et exécuter des tests en se basant directement sur le code source.

Elle a été appliquée principalement au **backend Spring Boot** à travers :

- des **tests unitaires JUnit**, ciblant la logique interne des services ;
- des **tests d'intégration**, vérifiant l'interaction entre les couches (service, configuration, sécurité, repository) ;
- la prise en compte de la **structure interne du code**, des branches conditionnelles et des cas d'exception.

Cette approche permet d'analyser la **qualité interne du code** et contribue directement à la mesure de la couverture via JaCoCo.

Tests boîte noire (Black Box Testing)

L'approche boîte noire a été utilisée pour tester le comportement fonctionnel de l'application :

- **tests Selenium UI**, validant les scénarios utilisateur depuis l'interface sans connaître le code ;
- **tests API fonctionnels**, vérifiant les réponses HTTP, messages JSON et comportements attendus.

Ces tests permettent d'assurer la **conformité fonctionnelle** et la qualité de l'expérience utilisateur.

2. Mesure de la couverture

Mesure de la couverture backend avec JaCoCo:

La couverture du code backend a été mesurée grâce à **JaCoCo**, intégré au cycle Maven. À l'exécution des tests, JaCoCo génère :

- un rapport XML (target/site/jacoco/jacoco.xml) utilisé par SonarQube ;

- un rapport HTML détaillé contenant :

  - ✓ les instructions exécutées
  - ✓ les branches (if/else) couvertes ou non
  - ✓ la complexité cyclomatique
  - ✓ le taux de couverture par classe ou par package.

Cette méthode permet de vérifier précisément quelles parties du code ont été testées par les tests unitaires.

Analyse et consolidation via SonarQube

SonarQube récupère les résultats de JaCoCo et présente plusieurs indicateurs de qualité :

- **coverage global** du backend ;
- **coverage sur nouveau code** ;
- **détection des classes non couvertes** ;

- **analyse des branches et conditions** ;
- **évaluation de la maintenabilité et de la fiabilité**.

SonarQube offre ainsi une vision globale et professionnelle de la qualité des tests.

Tests Selenium et couverture

Les tests Selenium sont **boîte noire** : ils ne génèrent **pas** de couverture de code, car ils testent l'application comme un utilisateur final. Ils contribuent à la **couverture fonctionnelle**, mais pas à la couverture technique (JaCoCo/LCOV).

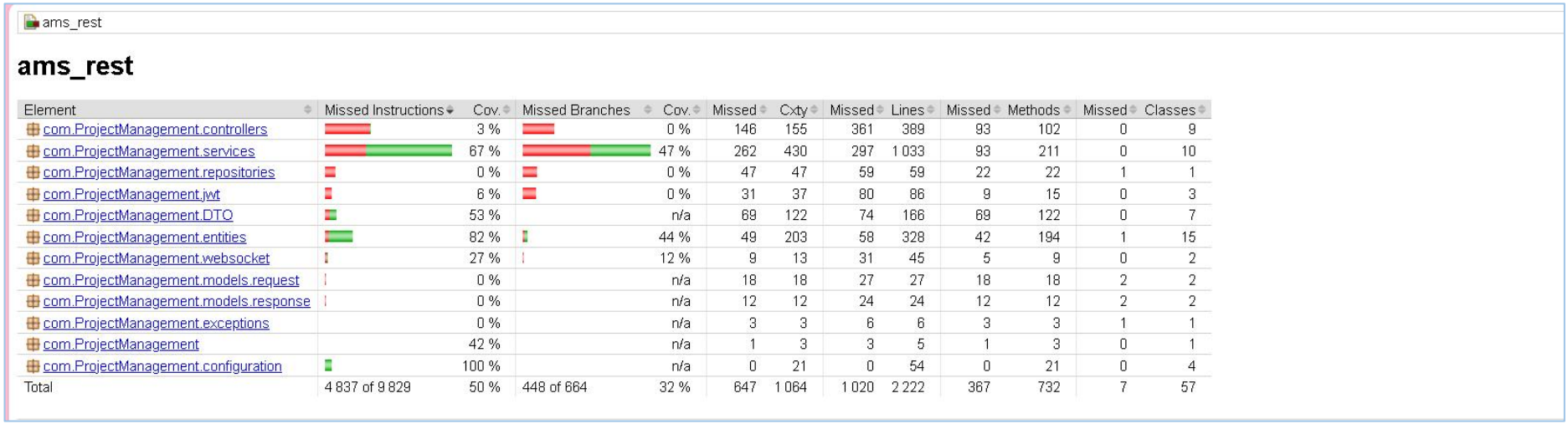
IV. Analyse des resultats Couverture des tests

L'analyse globale de la qualité du projet s'appuie sur plusieurs rapports générés automatiquement via le pipeline CI/CD :

- **JaCoCo** (couverture des tests boîte blanche)
- **SonarQube** (qualité du code, maintainability, duplications, reliability)
- **ExtentReport Selenium** (couverture fonctionnelle boîte noire)
- **Snyk** (analyse de sécurité)

Cette combinaison fournit une vision complète de la qualité logicielle : **interne**, **fonctionnelle**, et **sécuritaire**.

1. Analyse de la couverture du code (JaCoCo – White Box)



ams_rest

ams_rest

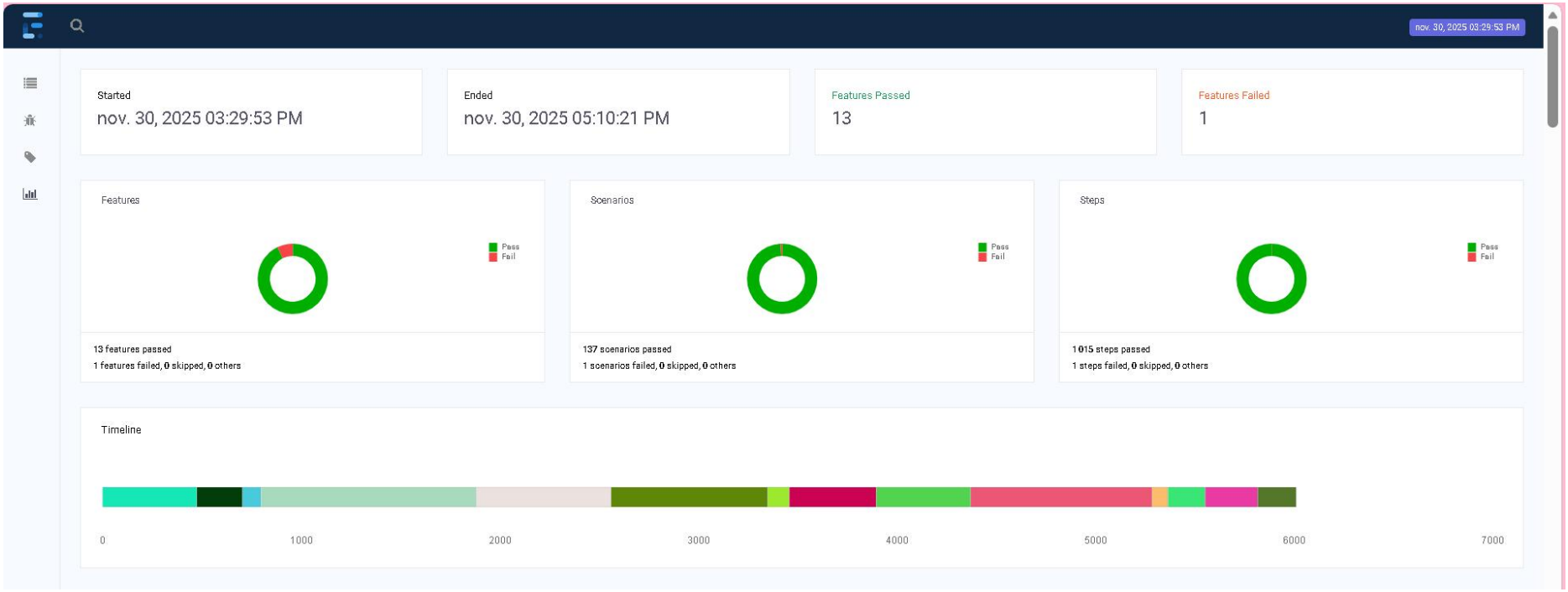
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.ProjectManagement.controllers		3 %		0 %	146	155	361	389	93	102	0	9
com.ProjectManagement.services		67 %		47 %	262	430	297	1 033	93	211	0	10
com.ProjectManagement.repositories		0 %		0 %	47	47	59	59	22	22	1	1
com.ProjectManagement.jwt		6 %		0 %	31	37	80	86	9	15	0	3
com.ProjectManagement.DTO		53 %		n/a	69	122	74	166	69	122	0	7
com.ProjectManagement.entities		82 %		44 %	49	203	58	328	42	194	1	15
com.ProjectManagement.websocket		27 %		12 %	9	13	31	45	5	9	0	2
com.ProjectManagement.models.request		0 %		n/a	18	18	27	27	18	18	2	2
com.ProjectManagement.models.response		0 %		n/a	12	12	24	24	12	12	2	2
com.ProjectManagement.exceptions		0 %		n/a	3	3	6	6	3	3	1	1
com.ProjectManagement		42 %		n/a	1	3	3	5	1	3	0	1
com.ProjectManagement.configuration		100 %		n/a	0	21	0	54	0	21	0	4
Total	4 837 of 9 829	50 %	448 of 664	32 %	647	1 064	1 020	2 222	367	732	7	57

Le rapport JaCoCo montre une couverture globale d'environ **50 % des instructions** et **32 % des branches**. L'analyse détaillée par package révèle :

nom de package	couverture en %	analyse et interprétation
Service	67	- Les services métier sont bien testés. -Cela montre une bonne stabilité côté logique métier et un faible risque de régression.
Entities	82	- Les entités sont fortement utilisées dans les tests, ce qui explique leur taux élevé
DTO	53	-Couverture acceptable liée à leur utilisation indirecte dans les scénarios métier.
Controllers	3%	Les endpoints REST ne sont presque pas couverts. Cela signifie que : - les validations HTTP - les réponses JSON - les erreurs possibles ne sont pas testées. C'est la zone la plus critique au niveau qualité.
JWT / Security	6%	Les filtres de sécurité ne sont pas testés (manque de tests MockMvc + Spring Security). Cela augmente le risque de failles ou de régressions dans la gestion des tokens.
Configuration	100%	Couvert automatiquement car exécuté au démarrage Spring Boot.

Le code est **partiellement bien testé**, avec un noyau métier stable, mais la couche Web (API REST), la sécurité et certaines branches restent insuffisamment couvertes.

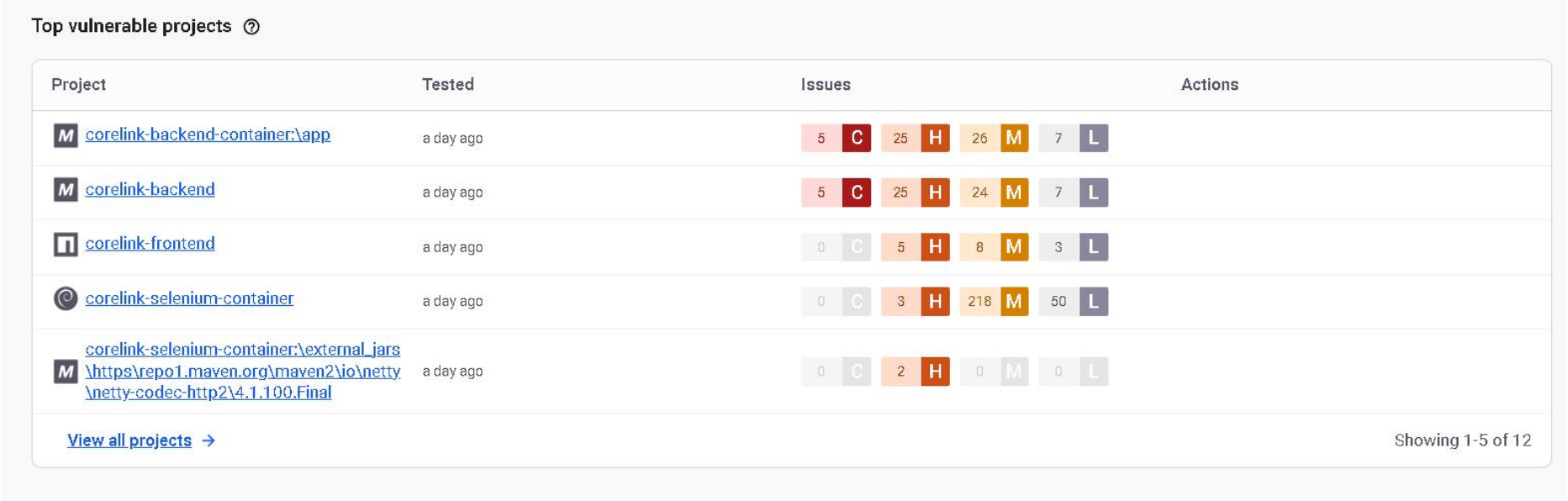
2. Analyse des tests fonctionnels (ExtentReport – Selenium – Black Box)



Les tests Selenium confirment que **la partie fonctionnelle est solide**, malgré un cas limite échoué.

Ils complètent parfaitement les tests boîte blanche en validant l’expérience utilisateur.

3. Analyse de la sécurité (Snyk)



Top vulnerable projects

Project	Tested	Issues	Actions
corelink-backend-container:\app	a day ago	5 C 25 H 26 M 7 L	
corelink-backend	a day ago	5 C 25 H 24 M 7 L	
corelink-frontend	a day ago	0 C 5 H 8 M 3 L	
corelink-selenium-container	a day ago	0 C 3 H 218 M 50 L	
corelink-selenium-container:\external_jars\https\repo1.maven.org\maven2\io\netty\netty-codec-http2\4.1.100.Final	a day ago	0 C 2 H 0 M 0 L	

View all projects →

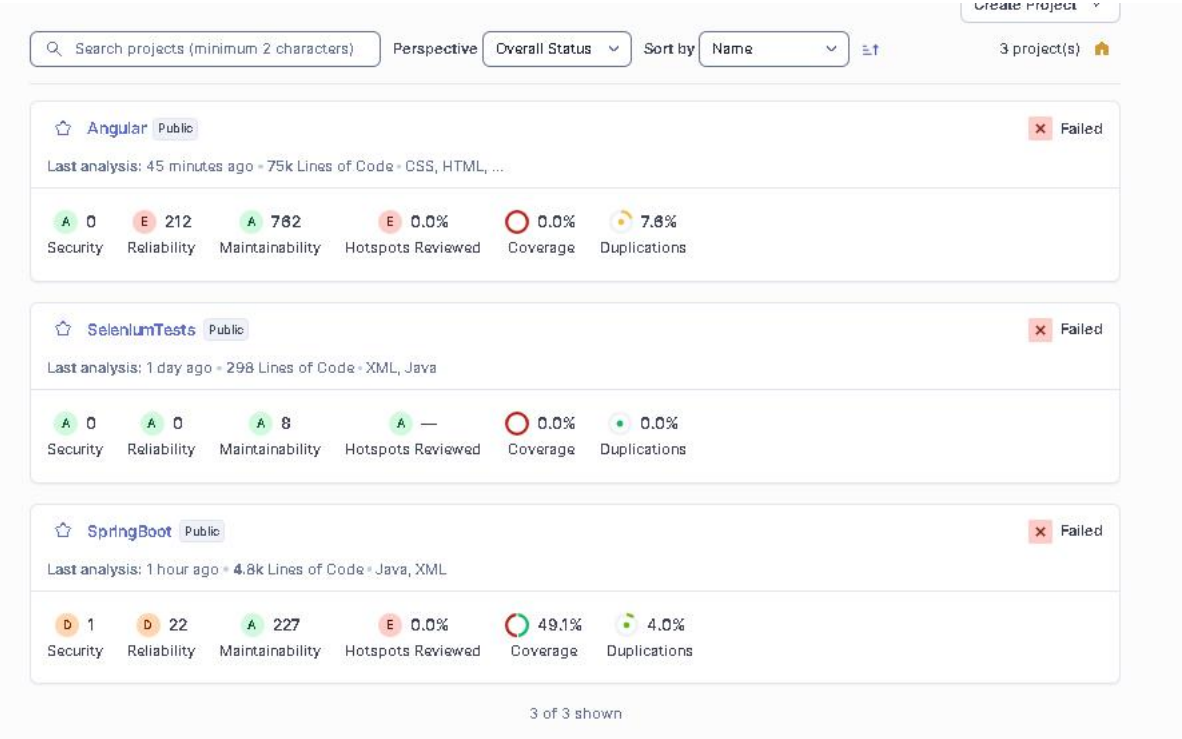
Showing 1-5 of 12

Les analyses Snyk révèlent que :

- Le backend (Spring Boot) représente la plus grande surface de vulnérabilité, avec plusieurs failles critiques provenant de dépendances Maven anciennes.

- Le frontend est relativement stable, avec uniquement quelques failles High et Medium liées à Node.js.
- Les conteneurs Selenium montrent un grand volume de vulnérabilités Medium, majoritairement issues des packages système d'images Docker officielles Selenium.

4. Analyse qualité SonarQube



- Le **backend est globalement fiable**, mais nécessite plus de tests REST et security.
- La **couche front-end** est maintenable mais manque de tests unitaires.
- Les **tests UI Selenium** montrent une bonne stabilité fonctionnelle.